

Database Systems (CS2005)

Week – 02

Instructor: **Basit Ali**

CHAPTER 2

Database System Concepts and Architecture

Data Models

- **Data Model**

- A set of concepts to describe the *structure* of a database, the *operations* for manipulating these structures, and certain *constraints* that the database should obey.

- **Data Model Structure and Constraints**

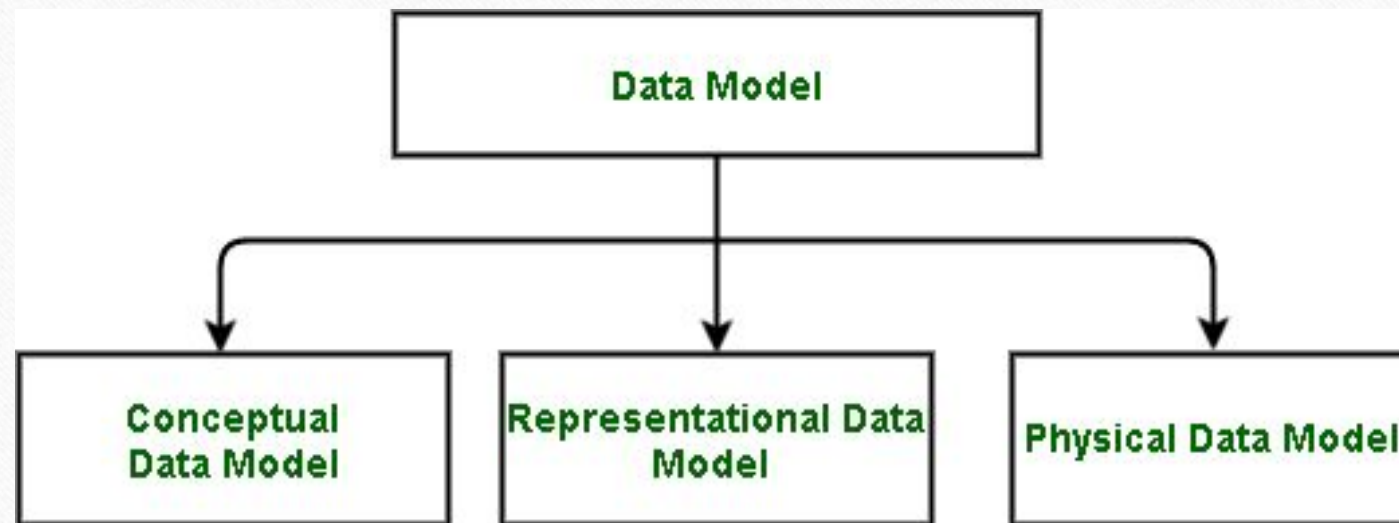
- Constructs are used to define the database structure
- Constructs typically include *elements* (and their *data types*) as well as groups of elements (e.g. *entity*, *record*, *table*), and *relationships* among such groups
- Constraints specify some restrictions on valid data; these constraints must be enforced at all times

Data Models (continued)

- **Data Model Operations**

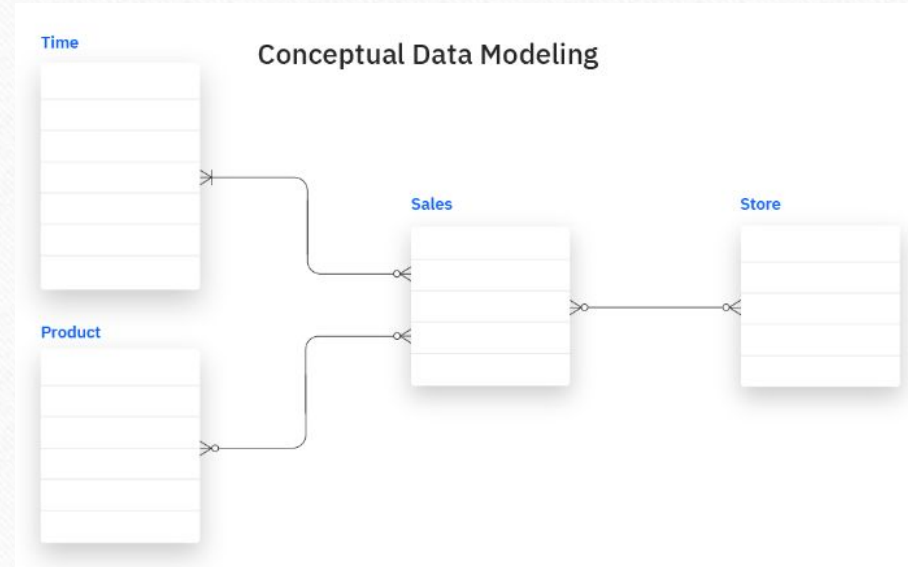
- These operations are used for specifying database *retrievals* and *updates* by referring to the constructs of the data model.
- Operations on the data model may include *basic model operations* (e.g. generic insert, delete, update) and *user-defined operations* (e.g. compute_student_gpa, update_inventory)

Categories of Data Models



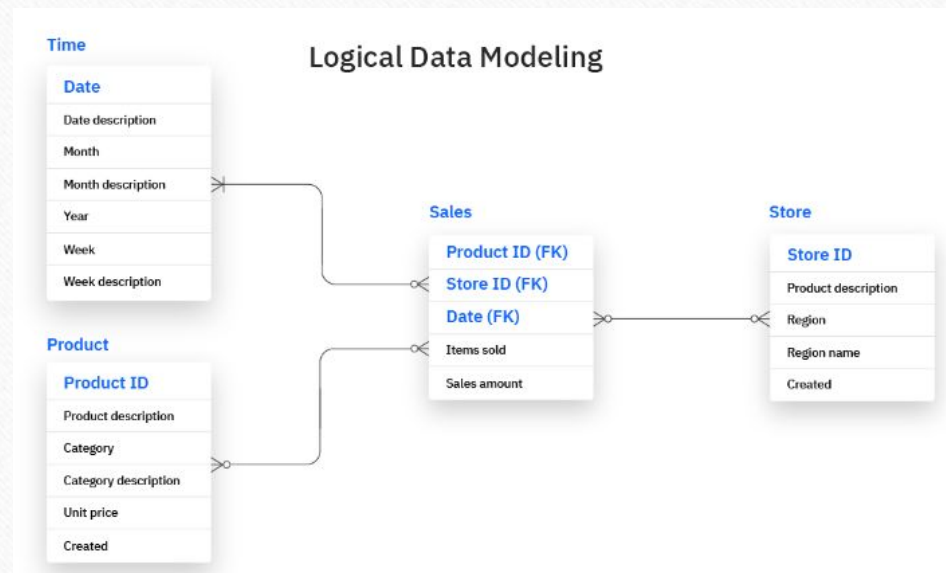
Conceptual data models

- They are also referred to as domain models and offer a big-picture view of what the system will contain, how it will be organized, and which business rules are involved.
- Conceptual models are usually created as part of the process of gathering initial project requirements.



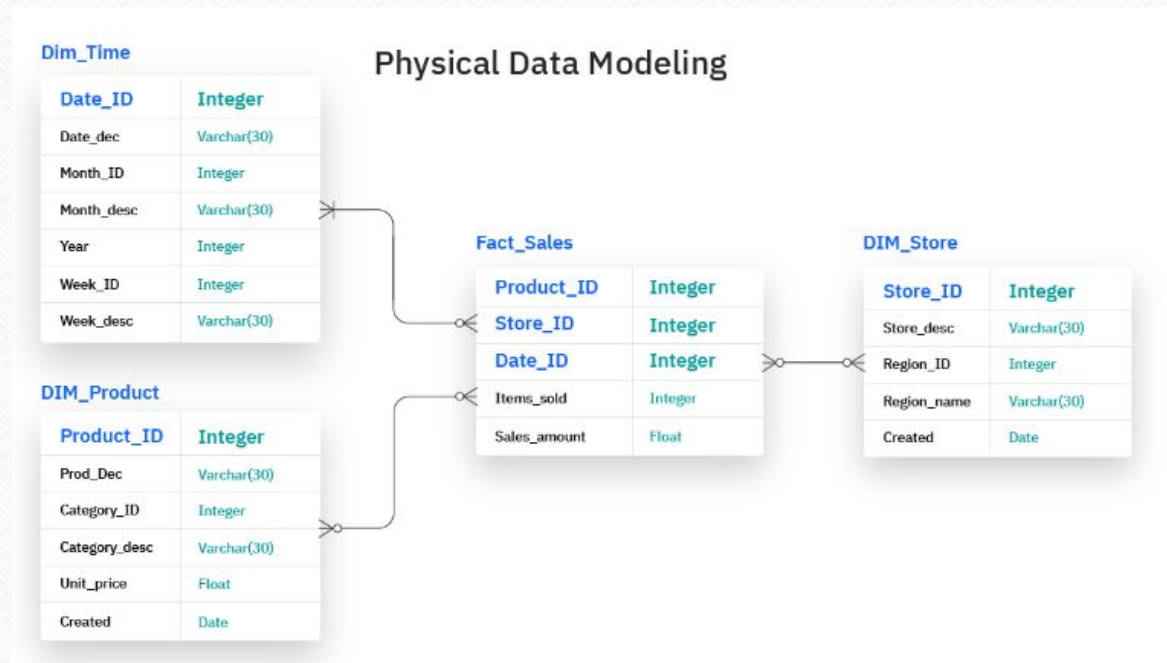
Representational/Logical Data Models

- They are less abstract and provide greater detail about the concepts and relationships in the domain under consideration.



Physical data models

- They provide a schema for how the data will be physically stored within a database.



Schemas versus Instances

- Database Schema
 - The *description* of a database.
 - Includes descriptions of the database structure, data types, and the constraints on the database.
- Schema Diagram
 - An *illustrative* display of (most aspects of) a database schema.
- Schema Construct
 - A *component* of the schema or an object within the schema, e.g., STUDENT, COURSE.

Schemas versus Instances

- Database State
 - The actual data stored in a database at a *particular moment in time*. This includes the collection of all the data in the database.
 - Also called database instance (or occurrence or snapshot).
 - The term *instance* is also applied to individual database components, e.g. *record instance*, *table instance*, *entity instance*

Database Schema vs. Database State

- Database State:
 - Refers to the *content* of a database at a moment in time.
- Initial Database State:
 - Refers to the database state when it is initially loaded into the system.
- Valid State:
 - A state that satisfies the structure and constraints of the database.

Database Schema vs. Database State (continued)

- Distinction
 - The *database schema* changes very infrequently.
 - The *database state* changes every time the database is updated.
- **Schema** is also called **intension**.
- **State** is also called **extension**.

Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

Example of a database state

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

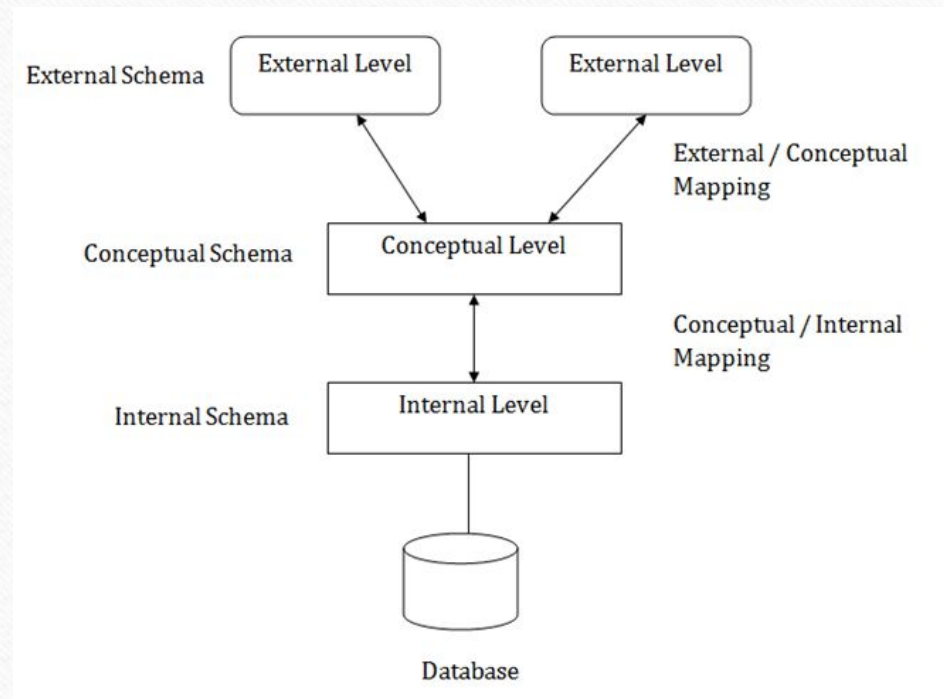
Figure 1.2

A database that stores student and course information.

Three schema Architecture

- The three schema architecture is also called ANSI/SPARC architecture or three-level architecture.
- This framework is used to describe the structure of a specific database system.
- The three schema architecture is also used to separate the user applications and physical database.
- The three schema architecture contains three-levels. It breaks the database down into three different categories.

Three schema Architecture



Three schema Architecture

- The main objective of three level architecture is to enable multiple users to access the same data with a personalized view while storing the underlying data only once.
- Different users need different views of the same data.
- The users of the database should not worry about the physical implementation and internal workings of the database such as data compression and encryption techniques, hashing, optimization of the internal structures etc.
- All users should be able to access the same data according to their requirements.

1. Internal Level

Internal view

STORED_EMPLOYEE record length 60

Empno : 4 decimal offset 0 unique
Ename : String length 15 offset 4
Salary : 8,2 decimal offset 19
Deptno : 4 decimal offset 27
Post : string length 15 offset 31

- The internal level has an internal schema which describes the physical storage structure of the database.
- The internal schema is also known as a physical schema.
- It uses the physical data model. It is used to define that how the data will be stored in a block.
- The physical level is used to describe complex low-level data structures in detail.

2. Conceptual Level

Global view

EMPLOYEE	
Empno	: Integer(4) Key
Ename	: String(15)
Salary	: String (8)
Deptno	: Integer(4)
Post	: String (15)

- The conceptual schema describes the design of a database at the conceptual level. Conceptual level is also known as logical level.
- The conceptual schema describes the structure of the whole database.
- The conceptual level describes what data are to be stored in the database and also describes what relationship exists among those data.

3. External Level



- An external schema is also known as view schema.
- Each view schema describes the database part that a particular user group is interested and hides the remaining database from that user group.
- The view schema describes the end user interaction with database systems.

Data Independence

- **Logical Data Independence**

- The capacity to change the conceptual schema without having to change the external schemas and their associated application programs.

- **Physical Data Independence**

- The capacity to change the internal schema without having to change the conceptual schema.
- For example, the internal schema may be changed when certain file structures are reorganized or new indexes are created to improve database performance

DBMS Languages

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
 - High-Level or Non-procedural Languages: These include the relational language SQL
 - May be used in a standalone way or may be embedded in a programming language
 - Low Level or Procedural Languages:
 - These must be embedded in a programming language

DBMS Programming Language Interfaces

- Programmer interfaces for embedding DML in a programming languages:
 - **Embedded Approach:** e.g. embedded SQL (for C, C++, etc.), SQLJ (for Java)
 - **Procedure Call Approach:** e.g. JDBC for Java, ODBC (Open Database Connectivity) for other programming languages as API's (application programming interfaces)
 - **Database Programming Language Approach:** e.g. ORACLE has PL/SQL, a programming language based on SQL; language incorporates SQL and its data types as integral components
 - **Scripting Languages:** PHP (client-side scripting) and Python (server-side scripting) are used to write database programs.

Typical DBMS Component Modules

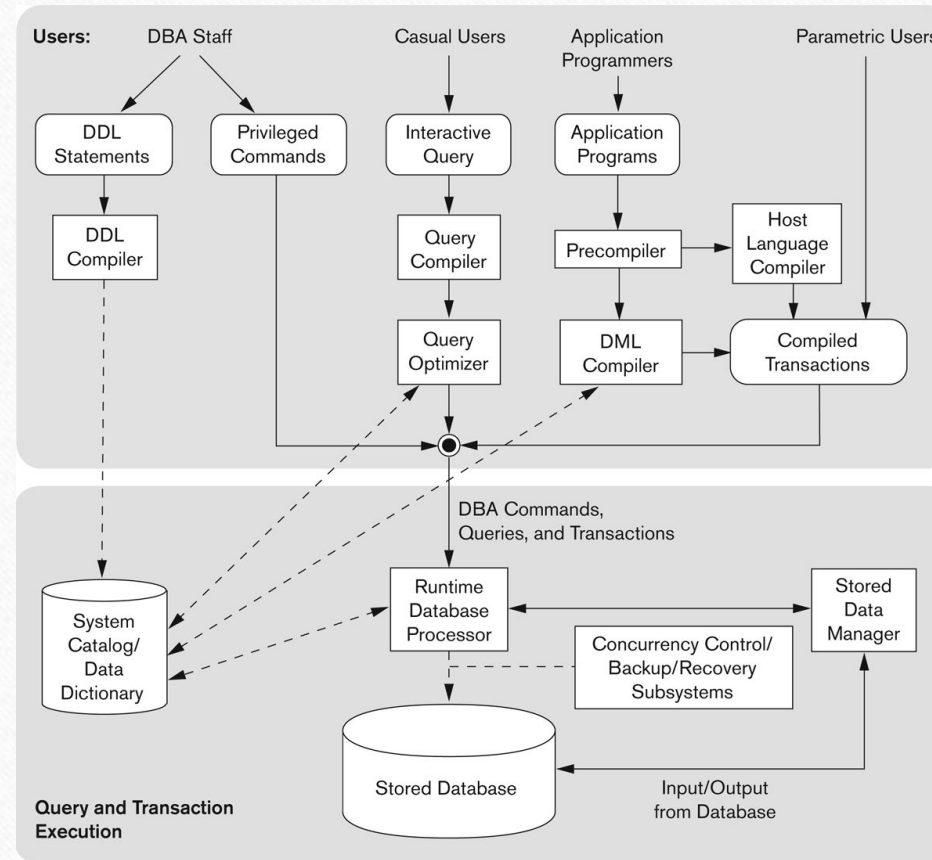


Figure 2.3

Component modules of a DBMS and their interactions.

Centralized and Client-Server DBMS Architectures

- Centralized DBMS
 - Combines everything into single system including- DBMS software, hardware, application programs, and user interface processing software.
 - User can still connect through a remote terminal – however, all processing is done at centralized site.

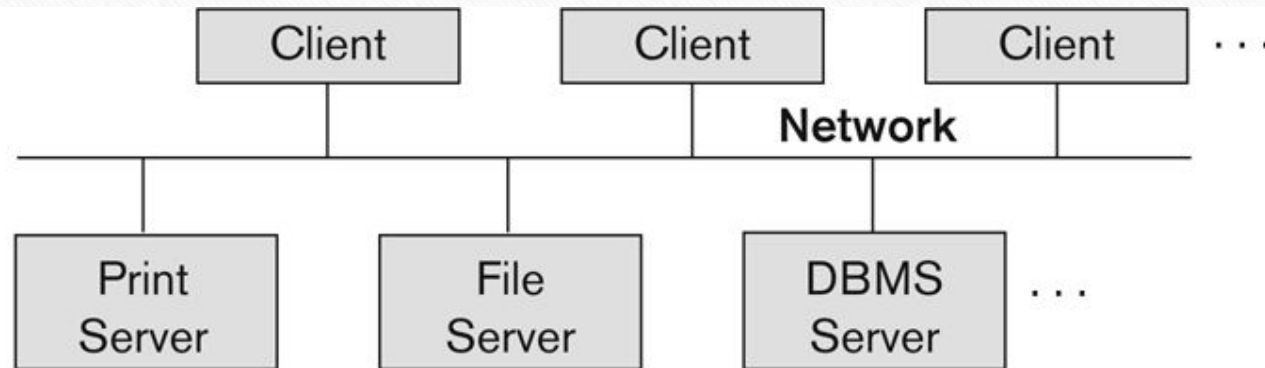
Basic 2-tier Client-Server Architectures

- Specialized Servers with Specialized functions
 - Print server
 - File server
 - DBMS server
 - Web server
 - Email server
- Clients can access the specialized servers as needed

Logical two-tier client server architecture

Figure 2.5

Logical two-tier
client/server
architecture.



Clients

- Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- Connected to the servers via some form of a network.
 - (LAN: local area network, wireless network, etc.)

DBMS Server

- Provides database query and transaction services to the clients
- Relational DBMS servers are often called SQL servers, query servers, or transaction servers
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
 - ODBC: Open Database Connectivity standard
 - JDBC: for Java programming access

Two Tier Client-Server Architecture

- Client and server must install appropriate client module and server module software for ODBC or JDBC
- A client program may connect to several DBMSs, sometimes called the data sources.
- In general, data sources can be files or other non-DBMS software that manages data.
- See Chapter 10 for details on Database Programming

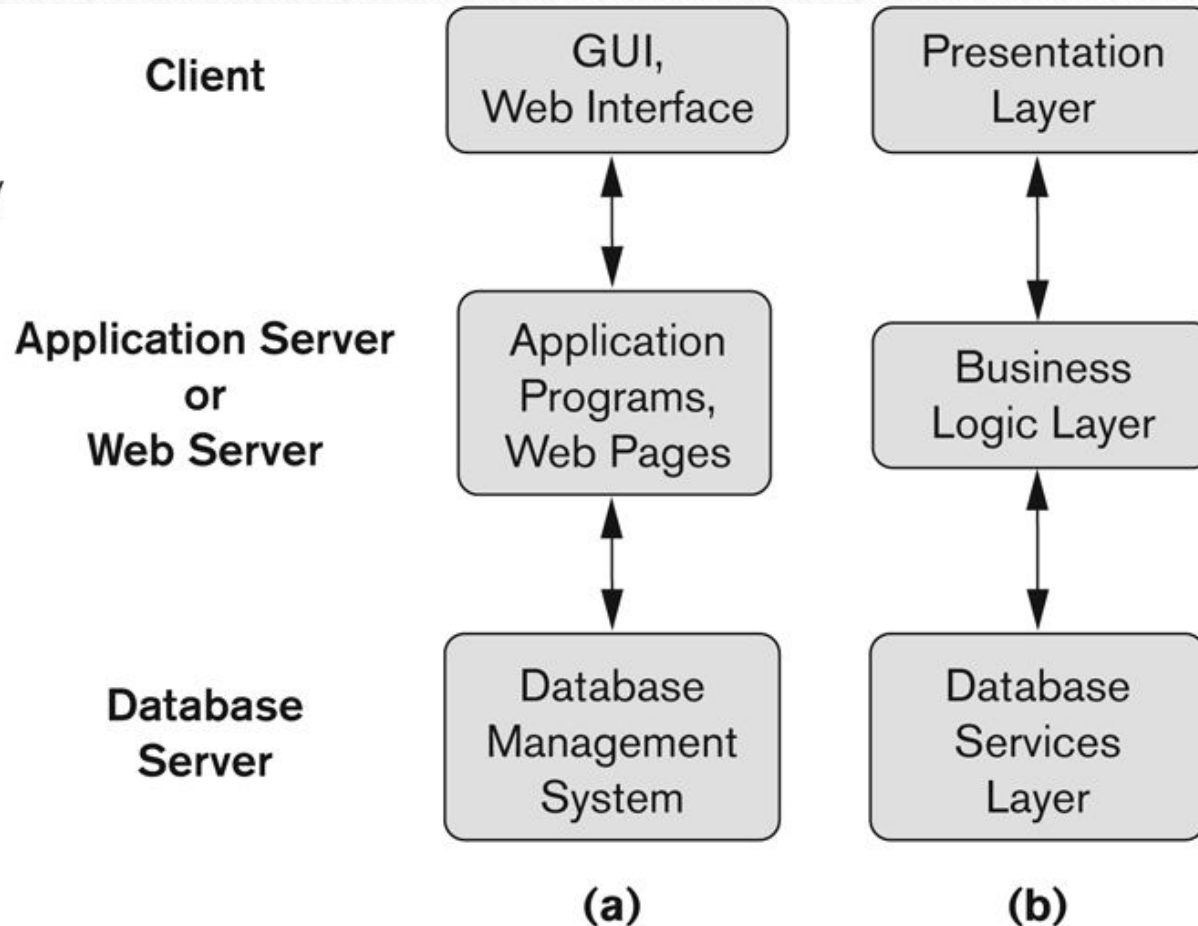
Three Tier Client-Server Architecture

- Common for Web applications
- Intermediate Layer called Application Server or Web Server:
 - Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
 - Acts like a conduit for sending partially processed data between the database server and the client.
- Three-tier Architecture Can Enhance Security:
 - Database server only accessible via middle tier
 - Clients cannot directly access database server
 - Clients contain user interfaces and Web browsers
 - The client is typically a PC or a mobile device connected to the Web

Three-tier client-server architecture

Figure 2.7

Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.



Variations of Distributed DBMSs (DDBMSs)

- Homogeneous DDBMS
- Heterogeneous DDBMS
- Federated or Multidatabase Systems
 - Participating Databases are loosely coupled with high degree of autonomy.
- Distributed Database Systems have now come to be known as client-server based database systems because:
 - They do not support a totally distributed environment, but rather a set of database servers supporting a set of clients.

History of Data Models

- **Hierarchical Data Model:**

- Initially implemented in a joint effort by IBM and North American Rockwell around 1965. Resulted in the IMS family of systems.
- IBM's IMS product had (and still has) a very large customer base worldwide
- Hierarchical model was formalized based on the IMS system
- Other systems based on this model: System 2k (SAS inc.)

Hierarchical Model

- Advantages:
 - Simple to construct and operate
 - Corresponds to a number of natural hierarchically organized domains, e.g., organization (“org”) chart
 - Language is simple:
 - Uses constructs like GET, GET UNIQUE, GET NEXT, GET NEXT WITHIN PARENT, etc.
- Disadvantages:
 - Navigational and procedural nature of processing
 - Database is visualized as a linear arrangement of records
 - Little scope for "query optimization"

History of Data Models

- **Relational Model:**

- Proposed in 1970 by E.F. Codd (IBM), first commercial system in 1981-82.
- Now in several commercial products (e.g. DB2, ORACLE, MS SQL Server, SYBASE, INFORMIX).
- Several free open source implementations, e.g. MySQL, PostgreSQL
- Currently most dominant for developing database applications.
- SQL relational standards: SQL-89 (SQL1), SQL-92 (SQL2), SQL-99, SQL3, ...
- Chapters 5 through 11 describe this model in detail

History of Data Models

- **Object-oriented Data Models:**
 - Several models have been proposed for implementing in a database system.
 - One set comprises models of persistent O-O Programming Languages such as C++ (e.g., in OBJECTSTORE or VERSANT), and Smalltalk (e.g., in GEMSTONE).
 - Additionally, systems like O2, ORION (at MCC - then ITASCA), IRIS (at H.P.- used in Open OODB).
 - Object Database Standard: ODMG-93, ODMG-version 2.0, ODMG-version 3.0.
 - Chapter 12 describes this model.

History of Data Models

- **Object-Relational Models:**

- The trend to mix object models with relational was started with Informix Universal Server.
- Relational systems incorporated concepts from object databases leading to object-relational.
- Exemplified in the versions of Oracle, DB2, and SQL Server and other DBMSs.
- Current trend by Relational DBMS vendors is to extend relational DBMSs with capability to process XML, Text and other data types.
- The term “Object-relational” is receding in the marketplace.